Assignment 8: Supervised Learning Classification

Classification of Titanic Passenger Survival Using Machine Learning

Random Forest achieved higher accuracy and better overall performance

- Key influential features:
  - Sex
  - Passenger Class (Pclass)
  - Fare

```
# 1. Import Libraries

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_s
```

```
# 2. Load Dataset

# Upload titanic.csv manually in Colab
df = pd.read_csv('/content/titanic.csv')
```

```
import os
print(os.listdir('/content'))
```

```
['.config', 'sample_data']
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files titanic.csv

titanic.csv(text/csv) - 61194 bytes, last modified: 4/7/2026 - 100% done

```
Saving titanic.csv to titanic.csv
```

```
import os
print(os.listdir('/content'))
```

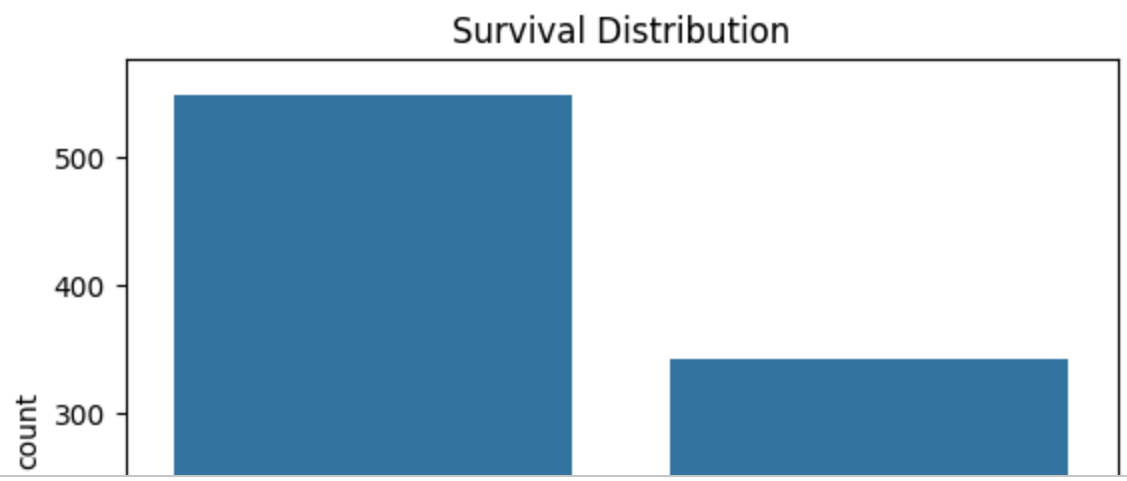
```
['.config', 'titanic.csv', 'sample_data']
```

```
df = pd.read_csv('/content/titanic.csv')
```

```
# 3. Data Preprocessing
# Handle missing values
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# Drop unnecessary columns
df.drop(['Name', 'Ticket', 'Cabin'], axis=1, inplace=True)

# Convert categorical variables
df = pd.get_dummies(df, drop_first=True)

# Features and target
X = df.drop('Survived', axis=1)
y = df['Survived']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

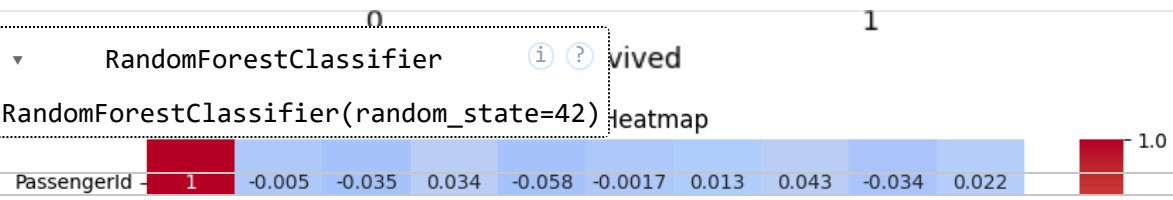
```
/tmp/ipykernel_11290/23546179.py:3: FutureWarning: A value is trying to be set c
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met


  df['Age'].fillna(df['Age'].median(), inplace=True)
/tmp/ipykernel_11290/23546179.py:4: FutureWarning: A value is trying to be set c
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met


  df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

```
# 4. Exploratory Data Analysis
sns.countplot(x='Survived', data=df)
plt.title("Survival Distribution")
plt.show()

plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

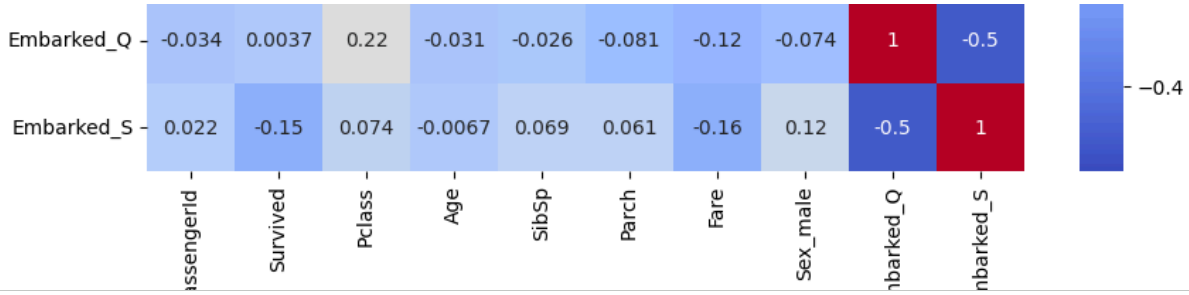



```
# 5. Model Training
# Logistic Regression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)

# Random Forest
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
```

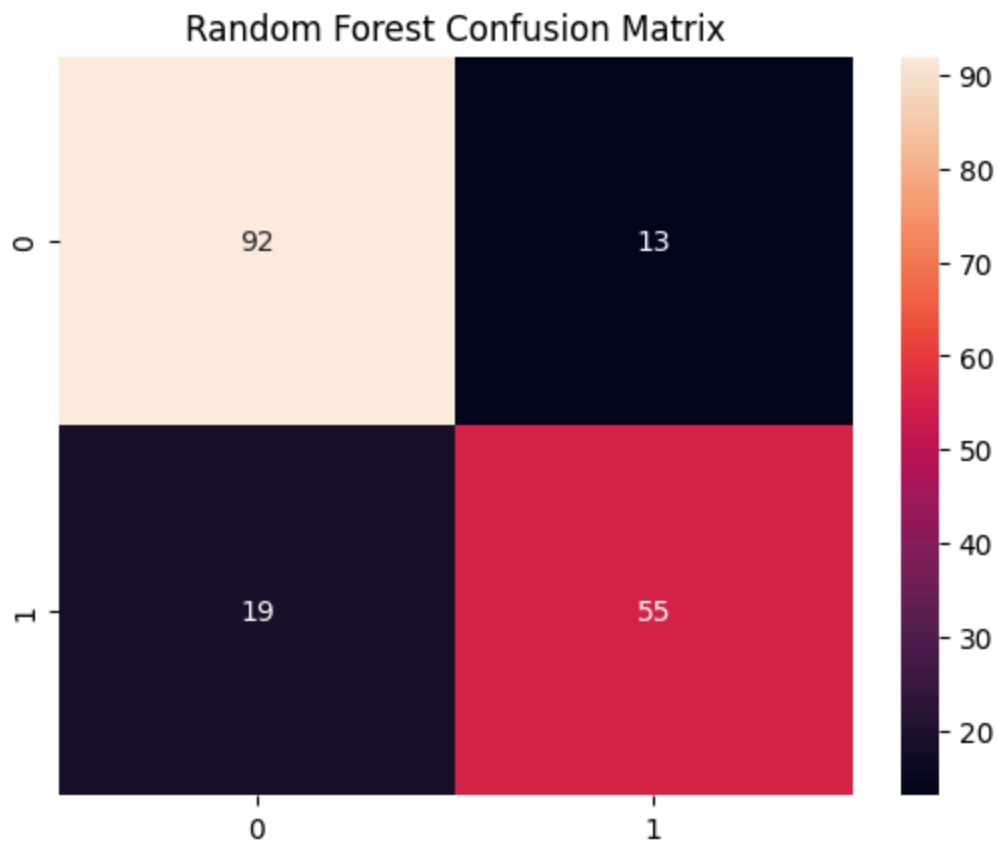
```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```




```
# 6. Evaluation

# Predictions
y_pred_lr = lr.predict(X_test)
y_pred_rf = rf.predict(X_test)

print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))

print("\nLogistic Regression Report:\n", classification_report(y_test, y_pred_l
print("\nRandom Forest Report:\n", classification_report(y_test, y_pred_rf))

# Confusion Matrix
sns.heatmap(confusion_matrix(y_test, y_pred_rf), annot=True, fmt='d')
plt.title("Random Forest Confusion Matrix")
plt.show()
```

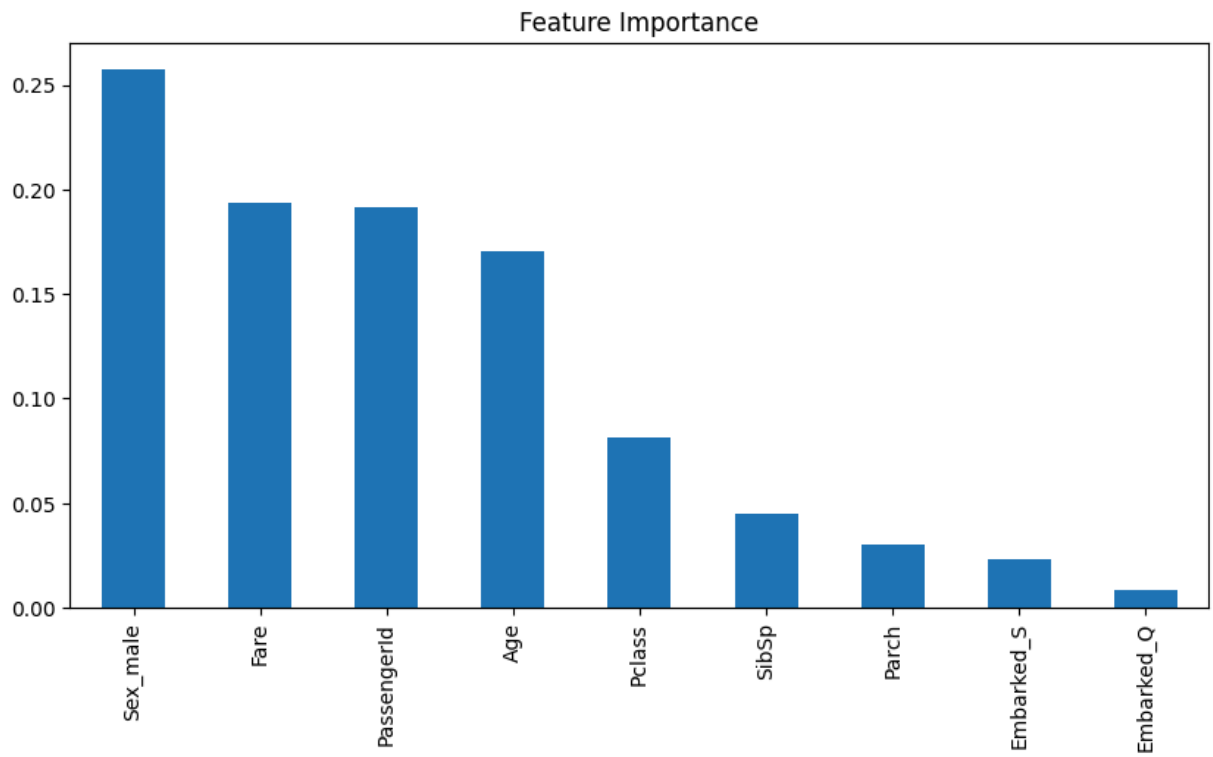


```
Logistic Regression Accuracy: 0.8044692737430168
Random Forest Accuracy: 0.8212290502793296

Logistic Regression Report:
              precision    recall  f1-score   support

           0       0.82      0.85      0.84       105
           1       0.77      0.74      0.76        74

    accuracy                           0.80       179
   macro avg       0.80      0.80      0.80       179
weighted avg       0.80      0.80      0.80       179


Random Forest Report:
              precision    recall  f1-score   support

           0       0.83      0.88      0.85       105
           1       0.81      0.74      0.77        74

    accuracy                           0.82       179
   macro avg       0.82      0.81      0.81       179
weighted avg       0.82      0.82      0.82       179
```




```
# 7. Feature Importance

importances = rf.feature_importances_
features = X.columns
```

```
feat_imp = pd.Series(importances, index=features).sort_values(ascending=False)

plt.figure(figsize=(10,5))
feat_imp.plot(kind='bar')
plt.title("Feature Importance")
plt.show()
```




```
# 8. Save Model

import joblib
joblib.dump(rf, 'titanic_model.pkl')
```

```
['titanic_model.pkl']
```